Jogo da Memória Online com Java RMI

Relatório do Trabalho 2 de Sistemas Distribuídos (versão compacta)

Campo	Valor
Aluno	Vinicius Alves Pereira
Matrícula	20223002747
Instituição e disciplina	CEFET, Sistemas Distribuídos
Código-fonte e scripts	Github
Demonstração	Demo - Trabalho 2
Aula	Aula - Trabalho 2

Resumo

Este relatório descreve um jogo da memória multijogador disputado online. Um middleware central em Java guarda a matriz secreta, o tabuleiro, os turnos e o placar, valida cada jogada e avisa todos os clientes por callbacks de Java RMI. Cada jogador usa um cliente Swing, em um processo próprio, que só recebe as cartas reveladas.

O texto descreve a arquitetura e a implantação (Registry, portas e `java.rmi.server.hostname`), o sorteio da matriz, as regras de turno e as validações, as decisões de concorrência e consistência (lock único, fila por jogador, sequência de estado e token de época), os contratos RMI e os DTOs, o reinício de partida com estatísticas de sessão e escolha do campeão, a política de falhas e desconexões, e as limitações conhecidas.

Sumário

- 1. Introdução
- 2. Tecnologias e ambiente
  - 2.1 Tecnologias e versões
- 3. Arquitetura e implantação
  - 3.1 Visão geral e diagrama de implantação
  - 3.2 Serviços no Registry
  - 3.3 Portas e `java.rmi.server.hostname`
  - 3.4 Publicação do middleware
  - 3.5 Lado cliente
  - 3.6 Fluxo de uma seleção de carta
- 4. Matriz, sorteio, imagens, turnos e validações
  - 4.1 Tabuleiro e posições
  - 4.2 Geração da matriz secreta
  - 4.3 Imagens e catálogo
  - 4.4 Turnos e regras de jogo
  - 4.5 Validação das jogadas
  - 4.6 Como o cliente apresenta o estado
- 5. Concorrência e consistência de estado
  - 5.1 Lock único e justo
  - 5.2 Fila e worker por jogador
  - 5.3 Sequência, versão e entrada em três fases
  - 5.4 Token de época dos temporizadores
  - 5.5 Callbacks e interface gráfica
  - 5.6 Garantias e limites
- 6. Contratos RMI e estruturas de dados
  - 6.1 Interfaces e exceções
  - 6.2 Operações
  - 6.3 Mapa de eventos
  - 6.4 DTOs
- 7. Sessão, reinício, aproveitamento e campeão

  - 7.1 Partida e sessão
  - 7.2 Fim da partida e continuar ou encerrar
  - 7.3 Reinício
  - 7.4 Tentativas e aproveitamento
  - 7.5 Sumário e campeão
- 8. Falhas e desconexões
  - 8.1 Detecção
  - 8.2 Política por fase
  - 8.3 Idempotência e isolamento
  - 8.4 Cliente diante de falhas
  - 8.5 Log
- 9. Conclusão, limitações e referências
  - 9.1 Conclusão
  - 9.2 Limitações
  - 9.3 Código-fonte
  - 9.4 Referências

---

1. Introdução

O problema tratado é um jogo da memória multijogador disputado online, em que vários processos independentes, um por jogador, precisam ver o mesmo estado, na mesma ordem, sem nunca conhecer a posição das cartas ainda fechadas. Isso exige uma autoridade central que guarde a matriz secreta, valide cada jogada, controle os turnos e o placar e avise todos os participantes, além de tratar jogadas simultâneas e desconexões sem bloquear os demais. O foco do trabalho é, portanto, invocação remota, coordenação centralizada, concorrência e consistência de estado, resolvidos com Java RMI. Ao longo do texto, Middleware designa o servidor central e Cliente designa o programa Swing de cada jogador.

---

2. Tecnologias e ambiente

2.1 Tecnologias e versões

Componente	Tecnologia	Versão	Uso
Linguagem	Java	17 (`maven.compiler.release`)	Todos os módulos (`record`, `switch` com expressões)
JDK para executar	JDK	17 ou 21	Comentário do `pom.xml` raiz
Build	Maven	3.x, com compiler 3.13.0 e shade 3.5.3	Build multimódulo e jars `middleware-all.jar` e `cliente-all.jar`
Comunicação e interface	Java RMI e Swing	Incluídos no JDK	Registry, stubs e callbacks; interface do cliente
Sorteio	`SecureRandom`	Incluído no JDK	Matriz (Fisher-Yates)
Concorrência	`ReentrantLock(true)`, `LinkedBlockingQueue`, `ScheduledExecutorService`	Incluídos no JDK	Lock justo da Sala, fila por jogador e temporizadores
Segurança de desserialização	`ObjectInputFilter`	Incluído no JDK	Lista de classes permitidas nos serviços
Bibliotecas de terceiros	Nenhuma	n/a	Os poms não declaram dependências externas nem de teste

Os fontes estão em UTF-8. `middleware` e `cliente` dependem só de `comum`, nunca um do outro. Não há `src/test`: o comportamento sob concorrência foi verificado por execuções manuais com vários clientes.

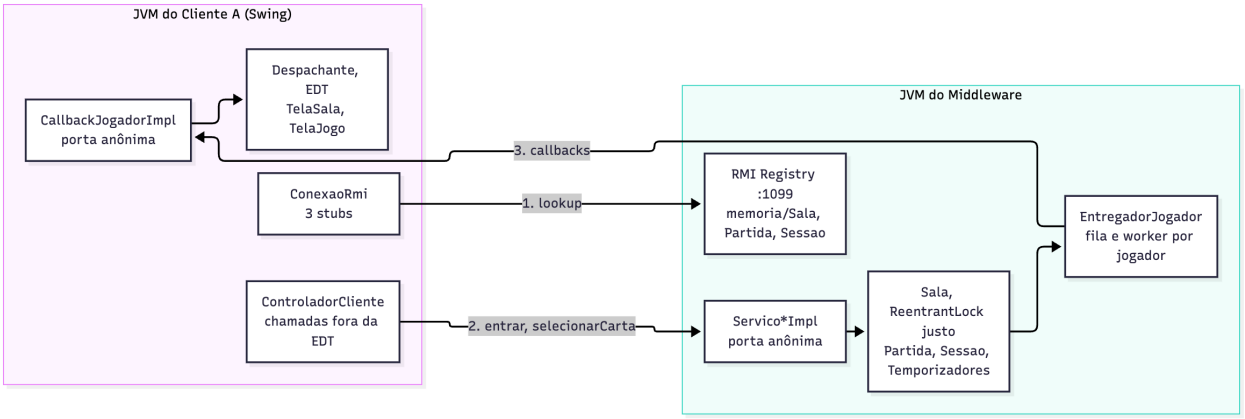
---

3. Arquitetura e implantação

Esta seção atende ao critério “Arquitetura Java, RMI, registro e callbacks” (20%).

3.1 Visão geral e diagrama de implantação

Há uma JVM do middleware (`java -jar middleware-all.jar`), que cria o RMI Registry no próprio processo, exporta três serviços e mantém a única Sala, e uma JVM por cliente (`cliente-all.jar`), com Swing, callback e conexões próprias. O módulo `comum` vai nos dois jars, então não há carregamento remoto de classes (nem `codebase`, nem gerenciador de segurança).



Os passos 2 e 3 também ocorrem com os clientes B, C e D. São dois sentidos de RMI: cliente para middleware (`LocateRegistry.getRegistry(host, porta)` e `lookup`) e middleware para cliente (o stub do `CallbackJogador` viaja como argumento de `ServicoSala.entrar`).

3.2 Serviços no Registry

Nome no Registry	Interface (`comum.remoto`)	Chamador e métodos
`memoria/Sala`	`ServicoSala`	Cliente: `entrar`, `sair`, `marcarPronto`, `notificarEncerramentoCliente`
`memoria/Partida`	`ServicoPartida`	Cliente: `selecionarCarta`, `consultarEstado`
`memoria/Sessao`	`ServicoSessao`	Cliente: `responderContinuar`
(não registrado)	`CallbackJogador`	Middleware: `ping` e os nove eventos ao... (seção 6.3)

O callback não vai ao Registry: o middleware só conhece os de quem entrou. Os nomes estão em `comum.remoto.NomesServico`.

3.3 Portas e `java.rmi.server.hostname`

O Registry usa a porta 1099 (padrão, escolhida no prompt do middleware e no campo Porta do cliente). Os objetos remotos do middleware e o callback do cliente usam porta anônima, sorteada pelo sistema operacional. O parâmetro `rmi.portaObjetos` existe, mas o `main` do middleware não o expõe. Em rede com firewall, o host do middleware precisa aceitar a porta do Registry e as efêmeras, e cada cliente precisa aceitar conexões de entrada do middleware, que abre a conexão do callback. Em `localhost` nada disso é necessário.

Um stub carrega o endereço onde o objeto escuta. Por padrão é o que a JVM descobre (pode ser `127.0.1.1`), o que funciona na mesma máquina e falha entre máquinas. Como há objetos exportados dos dois lados, o valor deve estar certo nos dois:

Ponta	Como o valor é definido
Middleware	Pergunta “IP deste host” (Enter aceita o IP detectado por `EntradaHost.detectarEnderecoLocal`). `PublicadorRmi.publicar` define a propriedade antes de qualquer exportação
Cliente	Campo “Meu IP” (vazio em teste local). Só vale na primeira tentativa de conexão do processo, porque o RMI lê a propriedade uma vez, e depois o campo fica somente leitura

O sintoma de valor errado é “funciona com `localhost`, não entre máquinas”.

3.4 Publicação do middleware

`Principal.main` ignora argumentos e pergunta IP, porta (1099), linhas (2) e colunas (3) do tabuleiro. A Sala usa valores fixos: capacidade 4, intervalo de erro de 2000 ms, inatividade de 60 s, prazo de “continuar” de 30 s, e timeout de callback e de `ping` de 5 s. A saída tem código 2 se a entrada acabar sem dados e código 1 se o Registry não puder ser publicado. A opção `-Dmemoria.log=caminho` grava o log em arquivo. A exportação e o registro seguem este trecho:

```
private void exportar(String nome, Remote impl, int portaObjetos)
        throws RemoteException {
    implementacoes.add(impl);   // referencia forte: evita GC
    Remote stub = UnicastRemoteObject.exportObject(impl, portaObjetos,
            null, null, FILTRO);
    registry.rebind(nome, stub);
    nomes.add(nome);
}
```

O `FILTRO` é um `ObjectInputFilter` de lista de permissões. Ele aceita `br.cefet.sd.memoria.comum.*`, `java.lang.*`, `java.util.*`, `java.math.*`, `java.rmi.*`, `sun.rmi.*`, primitivos, arrays e proxies de interfaces `Remote` permitidas (o stub do callback), e rejeita o resto. O desligamento (*shutdown hook*) faz `unbind` e desexporta os objetos.

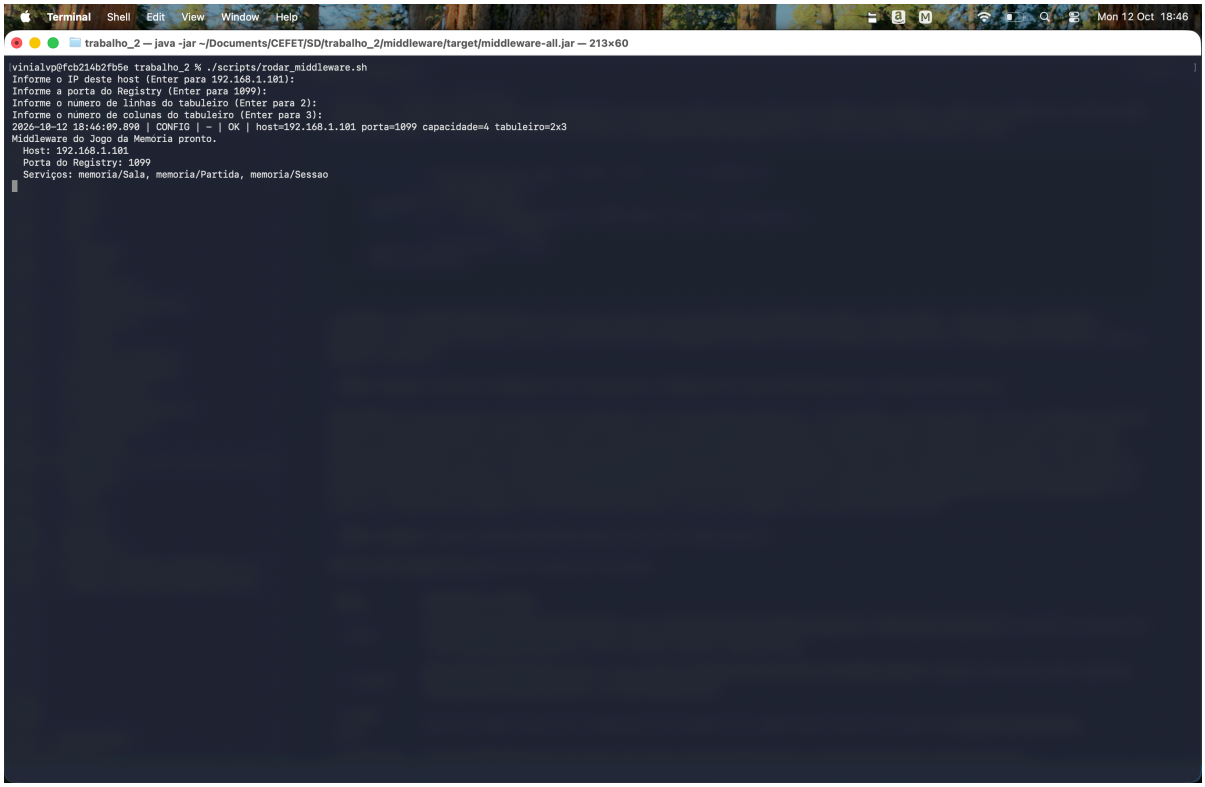


Figura 1: terminal do middleware com as perguntas de configuração e a lista dos três serviços publicados.

3.5 Lado cliente

O cliente não tem argumentos nem arquivo de configuração. A tela de conexão pede:

Campo	Padrão	Regra
Nome	vazio	1 a 20 caracteres na tela. O servidor valida 1 a 30 e a unicidade sem distinção de caixa
Host	`localhost`	Máquina do middleware
Porta	1099	Porta do Registry
Timeout	10 s	Tempo para localizar o middleware
Meu IP	vazio	Obrigatório entre máquinas (seção 3.3)

`ConexaoRmi.conectar` faz `getRegistry` e `lookup` dos três nomes em uma thread daemon, aguardada com o timeout. Qualquer falha vira `ConexaoRmiException` com host e porta, exibida na tela, sem encerrar o processo. Antes de entrar, o `ControladorCliente` exporta o callback (`UnicastRemoteObject.exportObject(this, 0)`), passa o stub a `ServicoSala.entrar` e o desexporta se a entrada for rejeitada.

Cada método do `CallbackJogadorImpl` retorna na hora: entrega o DTO a um `Despachante` (`SwingUtilities.invokeLater`). Assim a thread RMI do middleware nunca fica presa à interface, e a ordem de chegada é preservada pela fila FIFO da EDT.

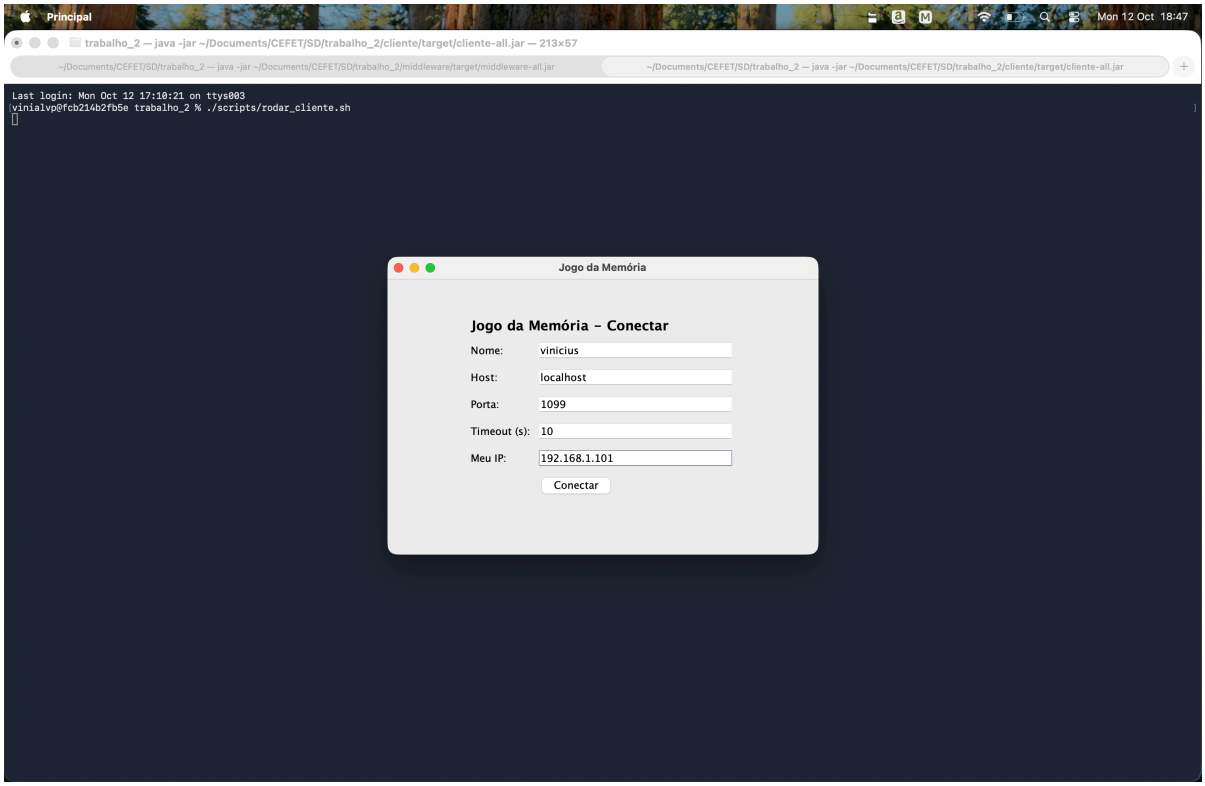


Figura 2: tela de conexão preenchida (Nome, Host, Porta, Timeout e Meu IP).

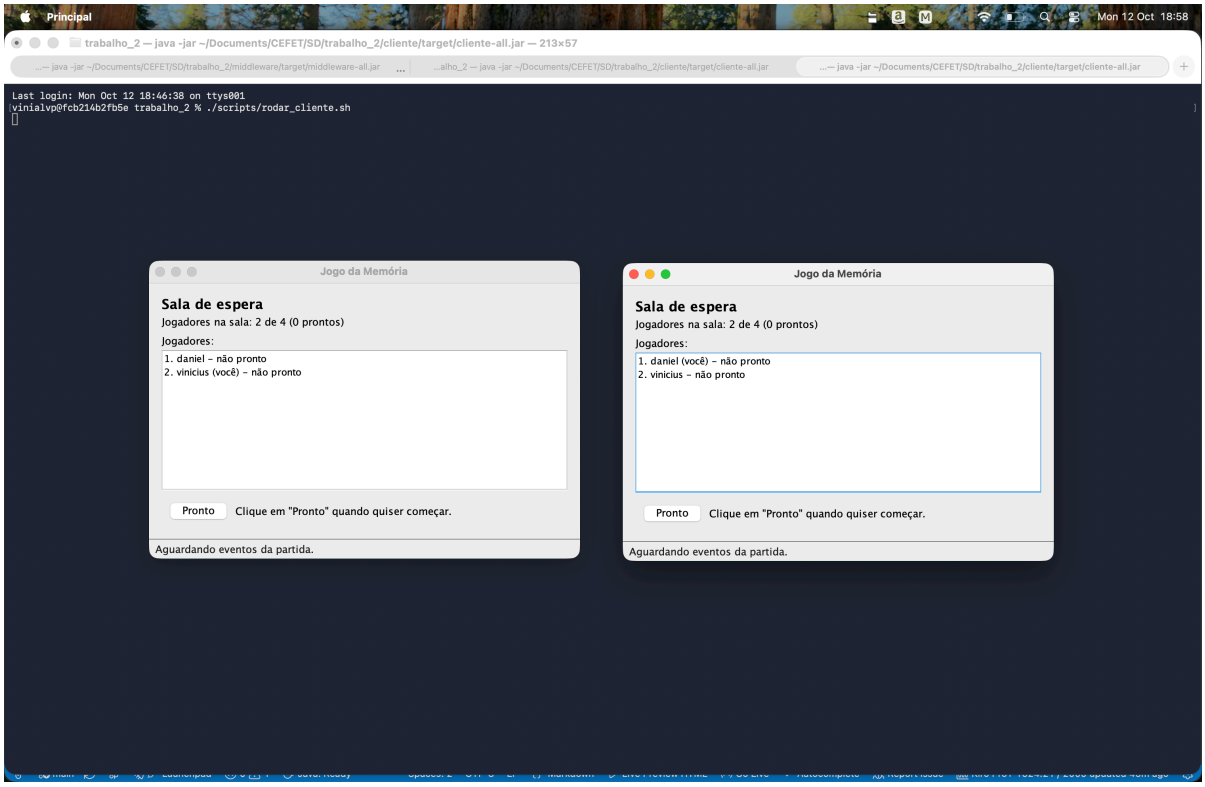


Figura 3: sala de espera com os jogadores daniel e vinicius, cada um vendo a própria entrada marcada com “(você)”.

3.6 Fluxo de uma seleção de carta

Etapa	Realização no código
1. Cliente	`ControladorCliente.selecionar` chama `selecionarCarta(idParticipante, idPartida, posicao)` fora da EDT, com prazo de 5 s. Não há atualização otimista: a carta só aparece quando o callback chega
2. Validação	`Sala.selecionarCarta` adquire o lock e delega a `Partida.selecionar` e `ValidadorJogada`. Em caso de rejeição nada muda e nada é publicado (`JogadaRejeitadaException` com `MotivoRejeicao`)
3. Estado oficial	Sob o lock, revela a carta e, na segunda, resolve par ou erro, placar e vez. Cada evento consome um número de `Sequencia_de_Estado`
4. Notificação	Os eventos são enfileirados sob o lock (O(1), sem rede) e entregues fora dele por um worker por jogador, com timeout de 5 s
5. Clientes	O callback vai à EDT, e o `ModeloCliente` descarta o que não for mais novo que o estado aplicado e atualiza tabuleiro, placar e vez

---

4. Matriz, sorteio, imagens, turnos e validações

Esta seção atende aos critérios “Matriz de pares, sorteio e imagens” (10%) e “Validação de jogadas, regras e turnos” (20%).

4.1 Tabuleiro e posições

O operador escolhe linhas e colunas no prompt, e o padrão é 2×3 (6 posições, 3 pares). `ConfiguracaoTabuleiro.validar` exige total par e maior ou igual a 4, e no máximo 12 pares (imagens do catálogo), logo de 4 a 24 posições. O prompt repete a pergunta até aceitar.

Entrada	Resultado
2×2 e 4×6	Aceitos
3×3 e 1×2	Recusados (NUMERO_DE_POSICOES_INVALIDO)
5×6	Recusado (IMAGENS_INSUFICIENTES)

A posição é um índice linear de base 0: `posicao = linha * colunas + coluna`. Na interface, a dica "Carta N" usa base 1. O tamanho vale até reiniciar o middleware.

As capturas deste relatório usam o tabuleiro padrão 2×3 (6 posições, 3 pares).

4.2 Geração da matriz secreta

`GeradorMatriz.gerar` monta `1,1,2,2,...` e embaralha só as posições (Fisher-Yates). Por isso nunca surge carta sem par nem identificador com mais de duas ocorrências. Em produção usa um novo `SecureRandom` a cada sorteio.

```
int[] m = new int[n];
for (int i = 0; i < n; i++) { m[i] = i / 2 + 1; }
for (int i = n - 1; i > 0; i--) {
    int j = rng.nextInt(i + 1);
    int t = m[i]; m[i] = m[j]; m[j] = t;
}
return m;
```

O construtor de `Partida` revalida o vetor (`validarDisposicao`: tamanho, intervalo `1..n/2`, no máximo duas ocorrências) e guarda uma cópia. `Sala.iniciarNovaPartida` é o único ponto que cria `Partida` (início a partir do lobby e reinício) e sorteia nova matriz a cada vez, sem comparar com a anterior (no 2×3 há 90 disposições). O log registra `INICIO_PARTIDA`, nunca a matriz.

4.3 Imagens e catálogo

`CatalogoImagens.carregar()` lê `catalogo.txt` (`id=arquivo`) e valida ids, nomes e repetições. Um tabuleiro de `n` posições usa os pares `1..n/2`. O `CatalogoImagensDTO` leva só `idPar -> arquivo`, sem posições, e os bytes das imagens não viajam por RMI, pois o cliente lê o PNG do próprio classpath. O catálogo chega em `EntradaSalaDTO` e nos estados `INICIO_PARTIDA` e `CONSULTA`.

A matriz nunca sai do núcleo. O campo `matriz` é `private final` em `Partida`, sem acessor. O estado público só preenche o `idPar` de cartas `REVELADA` ou `CONQUISTADA`, e o tipo reforça isso:

```
publicas.add(switch (cartas[i]) {
    case FECHADA -> new CartaPublicaDTO(EstadoCartaDTO.FECHADA, null);
    case REVELADA -> new CartaPublicaDTO(EstadoCartaDTO.REVELADA, matriz[i]);
    case CONQUISTADA -> new CartaPublicaDTO(EstadoCartaDTO.CONQUISTADA, matriz[i]);
});
// CartaPublicaDTO: lanca excecao se (estado == FECHADA) != (idPar == null)
```

Cada notificação é um *snapshot* completo. Durante o intervalo de erro as duas cartas têm `idPar` visível a todos e, ao fim, voltam como `FECHADA` sem `idPar`.

4.4 Turnos e regras de jogo

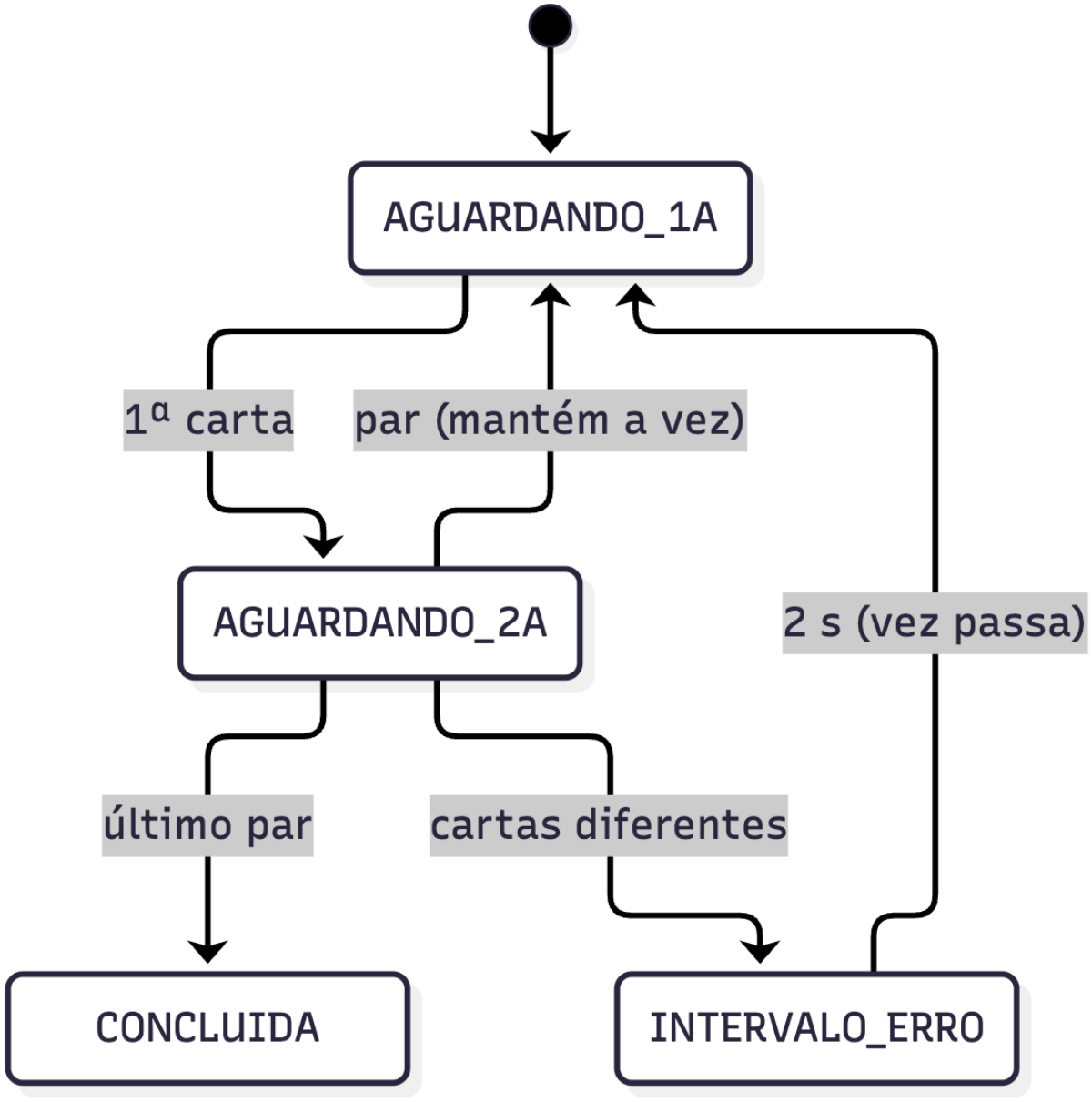
Primeiro jogador (regra documentada). O rodízio segue a ordem de entrada na Sala (`Sessao.escolherPrimeiro`). Na primeira partida começa quem entrou primeiro. Nas seguintes, começa o jogador de menor `ordemEntrada` estritamente maior que a do iniciador anterior, voltando ao início se não houver. Com Ana, Bruno e Carla em 4 partidas, a ordem é Ana, Bruno, Carla, Ana. A vez circula em ordem fixa (`indiceVez = (indiceVez + 1) % ordem.size()`), e a saída de qualquer jogador cancela a partida, então a lista não muda no meio dela.

Dinâmica.

- A primeira carta aceita só é revelada, sem tentativa. A segunda conta uma tentativa.
- Par: as duas viram `CONQUISTADA`, soma um par e mantém a vez.
- Erro: inicia o intervalo de erro (`INTERVALO_ERRO`, 2000 ms). As cartas ficam visíveis a todos e qualquer clique é rejeitado (`INTERVALO_DE_ERRO`). Ao fim, `fecharAposErro` fecha as cartas e passa a vez.

- Inatividade: 60 s sem jogar fecha a carta pendente, sem contar tentativa, e passa a vez.
- Fim: `cartasConquistadas == n` leva o status a `CONCLUIDA`.

Os sub-estados são `AGUARDANDO_1A` (nenhuma carta aberta), `AGUARDANDO_2A` (primeira aberta) e `INTERVALO_ERRO`. O status da partida é `EM_ANDAMENTO`, `CONCLUIDA` ou `CANCELADA`, distinto de `FaseSala` (`LOBBY`, `PARTIDA`, `COLETA_CONTINUAR`, `ENCERRADA`).



TipoEvento	Origem	Notificações, na ordem
`INICIO_PARTIDA`	`Partida.iniciar` (seq. 1)	`aoAtualizarTabuleiro` (com catálogo)
`CARTA_REVELADA`	Primeira carta	`aoAtualizarTabuleiro`
`PAR_ENCONTRADO` e `ERRO`	Segunda carta	`aoAtualizarTabuleiro`, `aoAtualizarPlacar`
`VEZ_PASSADA` e `INATIVIDADE`	Fim do intervalo ou 60 s	`aoAtualizarTabuleiro`, `aoMudarTurno`
`PARTIDA_CONCLUIDA`	Último par	tabuleiro, placar, `aoFinalizarPartida`, `aoPerguntarContinuar`

Núcleo da regra (`Partida.selecionar`, resumido). O validador e a mutação rodam sob o lock da Sala:

```
Optional<MotivoRejeicao> motivo = ValidadorJogada.validar(this, idParticipante,
        idPartida, posicao, disputouLock);
if (motivo.isPresent()) { return new ResultadoSelecao.Rejeitada(motivo.get()); }
epoca++;
cartas[p] = EstadoCarta.REVELADA;
if (sub == SubEstado.AGUARDANDO_1A) { primeira = p; sub = SubEstado.AGUARDANDO_2A; [...] }
segunda = p;
if (matriz[primeira] == matriz[segunda]) {      // acerto
    cartas[primeira] = cartas[segunda] = EstadoCarta.CONQUISTADA;
    cartasConquistadas += 2;
    contadores.put(jogador, contadores.get(jogador).comParEncontrado());
    if (cartasConquistadas == n) { status = StatusPartida.CONCLUIDA; [...] }
    [...]                                       // senao: mantem a vez
}
contadores.put(jogador, contadores.get(jogador).comErro());   // erro
sub = SubEstado.INTERVALO_ERRO;                 // vez passa no fim do intervalo
```

4.5 Validação das jogadas

O servidor valida em duas etapas, ambas sob o lock e antes de qualquer mutação. A primeira é `Sala.processarSelecao` (`SESSAO_ENCERRADA`, `PARTICIPANTE_INVALIDO`, `PARTIDA_ANTERIOR_OU_INEXISTENTE`). A segunda é `ValidadorJogada.validar`, função pura sobre `VisaoPartida`, que não expõe a matriz. A primeira regra violada define o único motivo devolvido:

```
if (idParticipante == null || !visao.participanteConhecido(idParticipante))
    return Optional.of(MotivoRejeicao.PARTICIPANTE_INVALIDO);
if (corrente.isEmpty() || idPartida == null || idPartida != corrente.getAsLong())
    return Optional.of(MotivoRejeicao.PARTIDA_ANTERIOR_OU_INEXISTENTE);
if (visao.status() != StatusPartida.EM_ANDAMENTO)
    return Optional.of(MotivoRejeicao.PARTIDA_ENCERRADA);
if (!idParticipante.equals(visao.idJogadorDaVez()))
    return Optional.of(MotivoRejeicao.FORA_DA_VEZ);
if (visao.subEstado() == SubEstado.INTERVALO_ERRO)
    return Optional.of(MotivoRejeicao.INTERVALO_DE_ERRO);
if (posicao == null || posicao < 0 || posicao >= visao.tamanho())
    return Optional.of(MotivoRejeicao.POSICAO_INVALIDA);
if (posicao == visao.primeiraPosicao())
    return Optional.of(disputouLock ? MotivoRejeicao.SELECAO_CONFLITANTE
                                    : MotivoRejeicao.SELECAO_REPETIDA);
if (visao.estadoCarta(posicao) == EstadoCarta.CONQUISTADA)
    return Optional.of(MotivoRejeicao.CARTA_INDISPONIVEL);
return Optional.empty();
```

Motivo	Quando	Alcançável pela interface?
`FORA_DA_VEZ`	Solicitante não é o da vez	Não (botões desabilitados), só por chamada RMI direta
`INTERVALO_DE_ERRO`	Clique durante os 2 s do erro	Sim, pois o cliente não conhece o sub-estado
`POSICAO_INVALIDA`	Nula, negativa ou maior ou igual a n	Não (chamada direta)
`CARTA_INDISPONIVEL`	Carta já conquistada	Não (botão desabilitado)
`SELECAO_REPETIDA` e	Mesma posição da primeira carta.	Só por chamadas simultâneas diretas
`SELECAO_CONFLITANTE`	“Conflitante” se a requisição esperou o lock	
`PARTIDA_ENCERRADA`, `PARTIDA_ANTERIOR_OU_INEXISTENTE`, `SESSAO_ENCERRADA`, `PARTICIPANTE_INVALIDO`	Estado ou id inválido	Não em uso normal

Toda rejeição deixa o estado intacto: nenhuma carta muda, nenhum evento é publicado, nenhuma sequência é consumida e os temporizadores não mudam. A exceção é lançada depois de liberar o lock. Pela ordem, `FORA_DA_VEZ` vem antes de intervalo e posição.

4.6 Como o cliente apresenta o estado

O cliente não conhece a matriz e não decide nada. `VisaoTabuleiro.de` habilita uma carta só se for a vez do jogador, a partida estiver `EM_ANDAMENTO`, não houver falha de conexão ou sincronização e a carta estiver `FECHADA`. O

`PainelTabuleiro` mostra `verso.png` para carta fechada e a imagem do catálogo para revelada ou conquistada (esta com borda verde e o texto “conquistada”, sem depender só de cor). A `TelaJogo` mostra “Sua vez”, “Aguarde sua vez (vez de X)” ou “Partida encerrada”, e `MensagensStatus` traduz os eventos (“X acertou um par!”, “X errou…”).

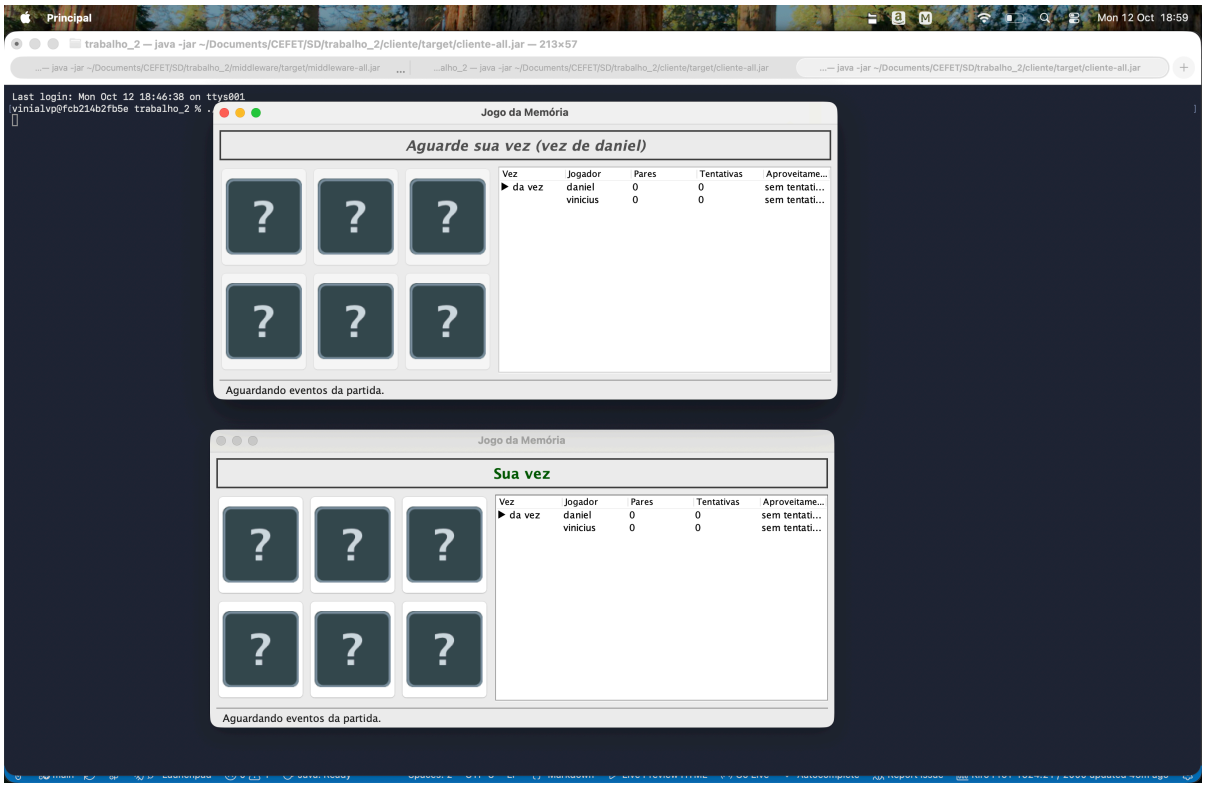


Figura 4: tabuleiro 2×3 fechado, com “Sua vez” para daniel (embaixo) e “Aguarde sua vez (vez de daniel)” para vinicius (em cima).

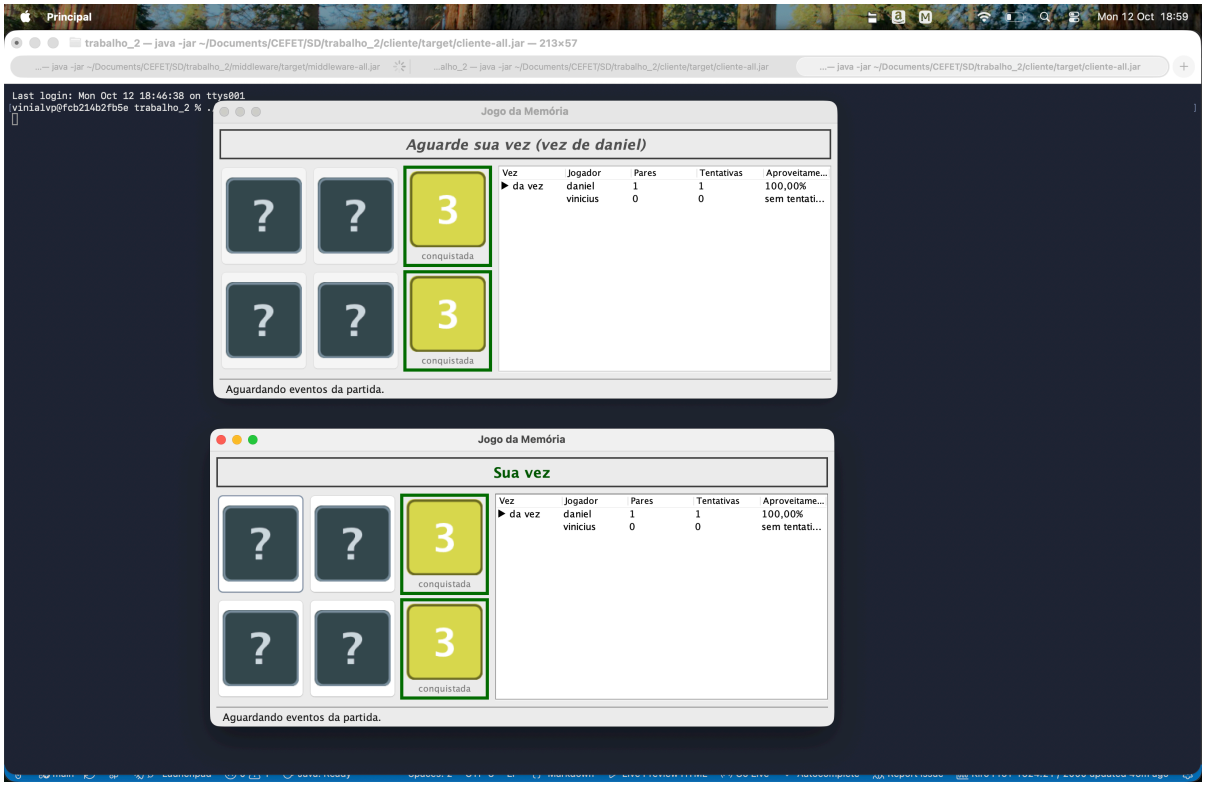


Figura 5: par de cartas “3” conquistado (borda verde e texto “conquistada”), placar de daniel com 1 par em 1 tentativa (100,00%) e a vez mantida com ele.

---

5. Concorrência e consistência de estado

Esta seção atende ao critério “Concorrência e tratamento de falhas remotas” (10%) e a parte de “Interfaces e sincronização” (15%). As chamadas RMI chegam em threads diferentes, além das dos temporizadores. Os riscos são jogadas conflitantes, par contado duas vezes, atualizações fora de ordem e um cliente lento travando a sala.

Problema	Mecanismo	Onde
Estado compartilhado	Um `ReentrantLock` justo por sala	`Sala.lock`
Seleções simultâneas	Serializadas pelo lock, cada uma validada contra o estado da anterior	`Sala.selecionarCarta`
I/O de rede lento	Eventos enfileirados sob o lock e entregues fora dele, com um worker por jogador	`DistribuidorEventos`, `EntregadorJogador`
Ordem das atualizações	`Sequencia_de_Estado` por partida, `versaoSala` por sala e fila FIFO por jogador	`Partida.consumirSequencia`, `Sala`
Notificação antiga no cliente	Descarte por `(idPartida, sequencia)`	`ModeloCliente.aceitar`
Temporizador obsoleto	Token de época	`Partida.epoca`, `Sala.epoca`

5.1 Lock único e justo

```
final ReentrantLock lock = new ReentrantLock(true);   // todo acesso ao estado oficial

public void selecionarCarta(String id, Long idPartida, Integer posicao)
        throws JogadaRejeitadaException {
    MotivoRejeicao rejeicao;
    boolean disputouLock = !lock.tryLock();
    if (disputouLock) { lock.lock(); }
    try { rejeicao = processarSelecao(id, idPartida, posicao, disputouLock); }
    finally { lock.unlock(); }
```

```
    if (rejeicao != null) { throw new JogadaRejeitadaException(rejeicao); }
}
```

- O lock é liberado uma vez, no `finally`, e a exceção sai fora da região crítica.
- O `tryLock()` sem argumentos pode furar a fila mesmo num lock justo. Isso relaxa a ordem estrita, não a exclusão mútua.
- `SELECAO_CONFLITANTE` aparece só para o jogador da vez em duas chamadas quase simultâneas à mesma posição, porque os outros jogadores caem antes em `FORA_DA_VEZ`. Duas chamadas do mesmo jogador a posições diferentes são ambas aceitas, como primeira e segunda carta, em sequência.
- Sob o lock só há trabalho de memória (validação, mutação, DTOs, enfileirar, agendar). Fora do lock ficam o `ping()` da entrada (até 5 s) e as chamadas de callback (até 5 s cada).

5.2 Fila e worker por jogador

Cada jogador tem um `EntregadorJogador`, com uma fila `LinkedBlockingQueue` e uma thread worker daemon. Publicar é percorrer os entregadores e enfileirar (O(1), sob o lock da Sala). O worker retira um evento por vez e chama o callback num pool compartilhado, esperando no máximo 5 s:

```
futuro = poolCallbacks.submit(() -> { evento.invocar(callback); return null; });
try {
    futuro.get(timeout.toNanos(), TimeUnit.NANOSECONDS);
    return true;
} catch (TimeoutException e)   { futuro.cancel(true); falhar(CausaDesconexao.TIMEOUT); return false; }
  catch (ExecutionException e) { falhar(CausaDesconexao.REMOTE_EXCEPTION); return false; }
```

O evento N+1 só sai depois que o N retornou ou falhou. Como o enfileiramento ocorre sob o lock, a ordem de cada fila é a ordem das mutações. Na falha (`RemoteException`, timeout ou `RuntimeException`), o entregador vai a `FALHOU`, descarta a fila e notifica a `Sala` uma única vez. O `aoFalhaCallback` roda fora do lock e só então o adquire para chamar `tratarDesconexao` (idempotente). Um callback travado prende apenas o worker daquele jogador.

5.3 Sequência, versão e entrada em três fases

Cada `Partida` tem `proximaSequencia` (começa em 1), e `consumirSequencia()` roda sob o lock. Os números são consecutivos, sem repetição e sem lacuna, pois rejeições não consomem. A pergunta de continuar e o sumário continuam a sequência da mesma partida. Os eventos de sala (`aoAtualizarSala` e `aoDesconectarJogador`) usam `versaoSala`, e `Sessao.proximoIdPartida` cresce de 1 em 1 na sessão.

`Sala.entrar` evita segurar o lock durante o `ping()` (rede, até 5 s):

1. `reservarVaga`, sob lock, valida fase, callback, capacidade, nome e unicidade (reservas contam como vaga e nome).
2. `EntregadorJogador.pingar`, fora do lock.
3. `confirmarEntrada`, sob lock, remove a reserva, rejeita com `CALLBACK_INVALIDO` se o ping falhou, reconfere a fase e só então cria o `Jogador` e publica a sala.

Assim um candidato com callback ruim não para a sala, e duas entradas não disputam a mesma vaga.

5.4 Token de época dos temporizadores

Há três temporizadores: fim do intervalo de erro, inatividade e prazo de continuar. Cancelar não basta, porque uma tarefa que já disparou pode estar esperando o lock. Cada agendamento captura a época vigente (`Partida.epoca`, e `Sala.epoca` para o prazo de continuar) e, ao disparar, a guarda compara com a atual e ignora a tarefa obsoleta:

```
public ResultadoTemporizador fecharAposErro(long epoca) {
    if (status != StatusPartida.EM_ANDAMENTO || epoca != this.epoca
            || sub != SubEstado.INTERVALO_ERRO) { return ResultadoTemporizador.IGNORADO; }
    [...]
}
```

A época avança em `iniciar()`, em cada seleção aceita, em cada passagem de vez e no `cancelar()`. Na Sala, ela avança ao iniciar ou cancelar a partida, abrir a coleta e encerrar a sessão. Exemplo: a inatividade armada na

época `k` dispara um instante antes de uma jogada. A jogada vence o lock, a época vira `k+1` e a tarefa antiga é ignorada, então a vez não passa a quem acabou de jogar.

5.5 Callbacks e interface gráfica

O cliente tem três tipos de thread: RMI (recebem callbacks), EDT (estado e interface) e as que chamam o servidor.

- **Descarte de antigas.** `ModeloCliente.aceitar` guarda o último `(idPartida, sequencia)`. Um `idPartida` menor é descartado, um maior é aceito e limpa os dados da partida anterior, e um igual exige `sequencia` maior. Eventos de sala comparam `versaoSala`.
- **Callback imediato e ordem.** O callback só empacota o DTO e chama `invokeLater`, e a EDT é FIFO.
- **Chamadas fora da EDT.** `ControladorCliente.executar` usa o `ExecutorRmi` (2 a 4 threads, fila de 256) e uma "vigia" que espera o `Future` com `get(prazo)`, e o resultado volta à EDT. Os prazos são 5 s (seleção, pronto, continuar, sair), 10 s (`entrar`) e 2 s (sincronização inicial e aviso de encerramento).
- **Modelo confinado à EDT.** `ModeloCliente` não tem `synchronized`, e só a EDT o toca. Assim "aplicar ou descartar" é atômico e o modelo é testável sem Swing.
- **Corrida na sincronização inicial.** Após `entrar`, o cliente chama `consultarEstado` (2 s). A consulta carrega os mesmos números dos eventos (sequência do último evento já emitido, sem consumir nova), então a ordem de chegada entre consulta e callbacks não importa: o mais antigo é descartado e o mais novo é aplicado.
- **Sem atualização otimista.** O clique não altera o modelo, e se o envio falha o tabuleiro permanece e aparece uma mensagem.

5.6 Garantias e limites

Garantia	Como
Sem jogadas conflitantes nem par duplicado	Validação e mutação sob um lock único, com o par apurado uma vez, na segunda carta aceita
Ordem por jogador	Enfileirar sob o lock, fila FIFO, um worker por jogador e sequência crescente
Cliente lento não bloqueia	Entrega fora do lock, por jogador, com timeout de 5 s e desconexão na falha
Cliente não aplica estado antigo	`(idPartida, sequencia)` e `versaoSala` no modelo confinado à EDT
Matriz nunca enviada	Campo privado, e o DTO público só tem `idPar` de cartas visíveis

Os limites são: uma sala por middleware, o lock único serializa tudo (aceitável para poucos jogadores) e a fila por jogador não tem limite de tamanho (sem contrapressão).

---

6. Contratos RMI e estruturas de dados

6.1 Interfaces e exceções

Todas as operações declaram `RemoteException`. As de domínio declaram também `ExcecaoDominio` (checked e serializável, com `CodigoErro`), exceto `notificarEncerramentoCliente`.

Exceção	Códigos
`EntradaRejeitadaException`	`PARTIDA_JA_COMECOU`, `SESSAO_EM_ANDAMENTO`, `CALLBACK_INVALIDO`, `SALA_CHEIA`, `NOME_INVALIDO`, `NOME_EM_USO`
`OperacaoInvalidaException`	`PARTICIPANTE_NAO_REGISTRADO`, `PARTICIPANTES_INSUFICIENTES`, `PARTIDA_EM_ANDAMENTO`, `SAIDA_NAO_PERMITIDA_EM_PARTIDA`, `SEM_PERGUNTA_ABERTA`, `JA_RESPONDEU`, `SESSAO_ENCERRADA`
`JogadaRejeitadaException`	Sempre `JOGADA_REJEITADA`, com `MotivoRejeicao`

Uma `RuntimeException` inesperada é traduzida por `TradutorErrosServico` em `RemoteException("Erro interno do servidor")`, e a pilha fica só no log do servidor.

```
public interface ServicoSala extends Remote {
    EntradaSalaDTO entrar(String nome, CallbackJogador callback)
            throws RemoteException, ExcecaoDominio;
    void sair(String idParticipante) throws RemoteException, ExcecaoDominio;
    void marcarPronto(String idParticipante) throws RemoteException, ExcecaoDominio;
    void notificarEncerramentoCliente(String idParticipante) throws RemoteException;
}
public interface ServicoPartida extends Remote {
    void selecionarCarta(String idParticipante, Long idPartida, Integer posicao)
            throws RemoteException, ExcecaoDominio;
    VisaoGeralDTO consultarEstado(String idParticipante)
            throws RemoteException, ExcecaoDominio;
}
public interface ServicoSessao extends Remote {
    void responderContinuar(String idParticipante, Boolean continuar)
            throws RemoteException, ExcecaoDominio;
}
public interface CallbackJogador extends Remote {
    void ping() throws RemoteException;
    void aoAtualizarSala(SalaDTO sala) throws RemoteException;
    void aoDesconectarJogador(DesconexaoDTO desconexao) throws RemoteException;
    void aoAtualizarTabuleiro(EstadoPublicoDTO estado) throws RemoteException;
    void aoMudarTurno(MudancaTurnoDTO mudanca) throws RemoteException;
    void aoAtualizarPlacar(PlacarDTO placar) throws RemoteException;
    void aoFinalizarPartida(ResultadoPartidaDTO resultado) throws RemoteException;
    void aoPerguntarContinuar(PerguntaContinuarDTO pergunta) throws RemoteException;
    void aoCancelarPartida(CancelamentoPartidaDTO cancelamento) throws RemoteException;
    void aoEncerrarSessao(SumarioSessaoDTO sumario) throws RemoteException;
}
```

`Long`, `Integer` e `Boolean` são *boxed* para o servidor validar `null`. O contrato do callback é retornar imediatamente.

6.2 Operações

Operação	Pré-condições	Efeitos	Exceções principais
`entrar`	Fase `LOBBY`, vaga e nome livre (1 a 30 pontos de código, sem distinção de caixa)	Cria jogador e entregador e publica `aoAtualizarSala` a todos. O `ping` (5 s) ocorre fora do lock	`EntradaRejeitadaException` (ordem: `PARTIDA_JA_COMECOU`, `SESSAO_EM_ANDAMENTO`, `CALLBACK_INVALIDO`, `SALA_CHEIA`, `NOME_INVALIDO`, `NOME_EM_USO`, ping falho)
`sair`	Id registrado, fase `LOBBY`	Remove o jogador e reavalia o início automático	`PARTICIPANTE_NAO_REGISTRADO`, `SAIDA_NAO_PERMITIDA_EM_PARTIDA`
`marcarPronto`	Id registrado, `LOBBY`, 2 ou mais jogadores	Marca pronto e, se todos estão prontos, inicia a partida. Idempotente no lobby	`PARTICIPANTE_NAO_REGISTRADO`, `PARTIDA_EM_ANDAMENTO`, `PARTICIPANTES_INSUFICIENTES`
`notificarEncerramentoCliente`	Nenhuma	No lobby equivale a `sair`. Nas demais fases vira desconexão `SAIDA_VOLUNTARIA`	Nunca lança `ExcecaoDominio`
`selecionarCarta`	Regras da seção 4.5	Retorno `void`, com efeito só por callback	`JogadaRejeitadaException`
`consultarEstado`	Id registrado	Somente leitura: não consome sequência nem versão, e `partida` é nula no lobby	`PARTICIPANTE_NAO_REGISTRADO`
`responderContinuar`	Fase `COLETA_CONTINUAR`, ainda não respondeu	A primeira resposta prevalece. “Não” encerra já e “sim” aguarda os demais	`SESSAO_ENCERRADA`, `PARTICIPANTE_NAO_REGISTRADO`, `SEM_PERGUNTA_ABERTA`, `JOGADA_REJEITADA` (resposta nula), `JA_RESPONDEU`

6.3 Mapa de eventos

Ordem entregue a cada jogador, em que `V` é `versaoSala` e `s` é a sequência.

Mutação	Eventos
Entrada, saída no lobby ou pronto	`aoAtualizarSala(V)`, que pode iniciar a partida
Início de partida	`aoAtualizarTabuleiro` (`INICIO_PARTIDA`, seq. 1, com catálogo). Não há `aoAtualizarSala` nem `aoMudarTurno`
Primeira carta	`aoAtualizarTabuleiro(s)`
Par ou erro	`aoAtualizarTabuleiro(s)` e `aoAtualizarPlacar(s+1)`. Par mantém a vez, sem `aoMudarTurno`
Fim do intervalo ou inatividade	`aoAtualizarTabuleiro(s)` e `aoMudarTurno(s+1)`
Último par	Tabuleiro `PARTIDA_CONCLUIDA`, placar, `aoFinalizarPartida`, `aoPerguntarContinuar` e depois `aoAtualizarSala(V)` com `COLETA_CONTINUAR`
Desconexão em partida, 2 ou mais restantes	`aoDesconectarJogador`, `aoAtualizarSala`, `aoCancelarPartida`, `aoPerguntarContinuar`, `aoAtualizarSala`
Desconexão em partida, 1 restante	`aoDesconectarJogador`, `aoAtualizarSala`, `aoCancelarPartida`, `aoEncerrarSessao`
"Não" ou prazo esgotado	`aoEncerrarSessao`, sem `aoAtualizarSala`
Todos "sim"	`aoAtualizarTabuleiro` (`INICIO_PARTIDA`, novo `idPartida`, seq. 1)

Cada evento é entregue uma vez, em ordem, a quem estava registrado na publicação. Não há reenvio: quem falha é desconectado, e a recuperação fica por conta de `consultarEstado`.

6.4 DTOs

Os DTOs ficam no pacote `comum.dto`: são 16 `record` imutáveis e `Serializable` (`serialVersionUID = 1L`), com listas copiadas por `List.copyOf`. O construtor compacto valida invariantes. Como o `record` é desserializado pelo construtor canônico, um fluxo adulterado que viole um invariante é recusado.

DTO	Campos principais	Invariantes
`SalaDTO`, `ParticipanteDTO`	Versão, participantes (id, nome, pronto, ordem), capacidade e fase	`versao >= 0`, participantes até a capacidade
`EntradaSalaDTO`	`idParticipante`, catálogo e sala	Sem nulos
`CatalogoImagensDTO`	`Map<Integer,String>` de par para arquivo	Chaves `1..N`, arquivos distintos, sem posições
`CartaPublicaDTO`	Estado e `idPar`	`idPar` nulo se e somente se `FECHADA`
`EstadoPublicoDTO`	`idPartida`, `sequencia`, linhas, colunas, cartas, ordem, vez, placar, status, último evento e catálogo	`cartas.size() == linhas*colunas`. Vez e catálogo podem ser nulos
`PlacarJogadorDTO`	Pares, tentativas, `BigDecimal` de aproveitamento e `semTentativas`	`pares <= tentativas`, aproveitamento de 0 a 100
`MudancaTurnoDTO`, `PlacarDTO`, `PerguntaContinuarDTO`	`idPartida`, `sequencia` e dados do evento	`sequencia >= 1`
`ResultadoPartidaDTO`	`empate`, vencedores, placar final e prazo	`empate` se e somente se 2 ou mais vencedores
`CancelamentoPartidaDTO`, `DesconexaoDTO`	Nome do jogador e `CausaDesconexao` (`REMOTE_EXCEPTION`, `TIMEOUT`, `SAIDA_VOLUNTARIA`)	Nome e causa não nulos
`SumarioSessaoDTO`, `SumarioJogadorDTO`	Concluídas, canceladas, empates, jogadores e campeões	Contagens maiores ou iguais a 0
`VisaoGeralDTO`	Sala e partida (pode ser nula)	Sala não nula

Nunca vão ao cliente a matriz secreta, o `idPar` de cartas fechadas e os detalhes internos de erro. Uma limitação é que os ids (UUID) de todos os participantes vão a todos os clientes, o que é necessário para comparar a vez. Sem autenticação, um cliente modificado poderia agir por outro jogador (seção 9.2).

DTOs, exceções e tipos simples passam por valor, e só `Remote` (os três serviços e o callback) passa por referência (stub). O filtro vale para o que o cliente envia ao servidor. O callback do cliente não define filtro próprio e depende da validação dos construtores.

7. Sessão, reinício, aproveitamento e campeão

Esta seção atende a “Fim de partida, reinício, sumário e campeão” (15%). Tudo é calculado no middleware, e o cliente só exibe.

7.1 Partida e sessão

`Partida` guarda matriz, cartas, vez, sub-estado, épocas, placar da partida e `Sequencia_de_Estado`. `Sessao` (classe pura, protegida pelo lock da Sala) guarda partidas concluídas e canceladas, vitórias, pares e tentativas acumulados por jogador, empates e o iniciador anterior. O reinício cria nova `Partida` e mantém a `Sessao`, e encerrar descarta as duas.

7.2 Fim da partida e continuar ou encerrar

`ApuracaoPartida.vencedores` devolve todos os jogadores com o máximo de pares: com 1 há um vencedor, com 2 ou mais há empate (sem vitória individual). Com 2 jogadores num 2×3 o empate é impossível (3 pares), e ocorre, por exemplo, com 3 jogadores no 2×3. O validador passa a rejeitar tudo com `PARTIDA_ENCERRADA`. `Sala.aoPartidaConcluida` registra o resultado na `Sessao` e abre a coleta. O cliente mostra o diálogo não modal “Resultado da partida” (vencedor ou empate e tabela “Placar final”), com os botões “Continuar jogando” e “Encerrar sessão” habilitados quando a pergunta chega.

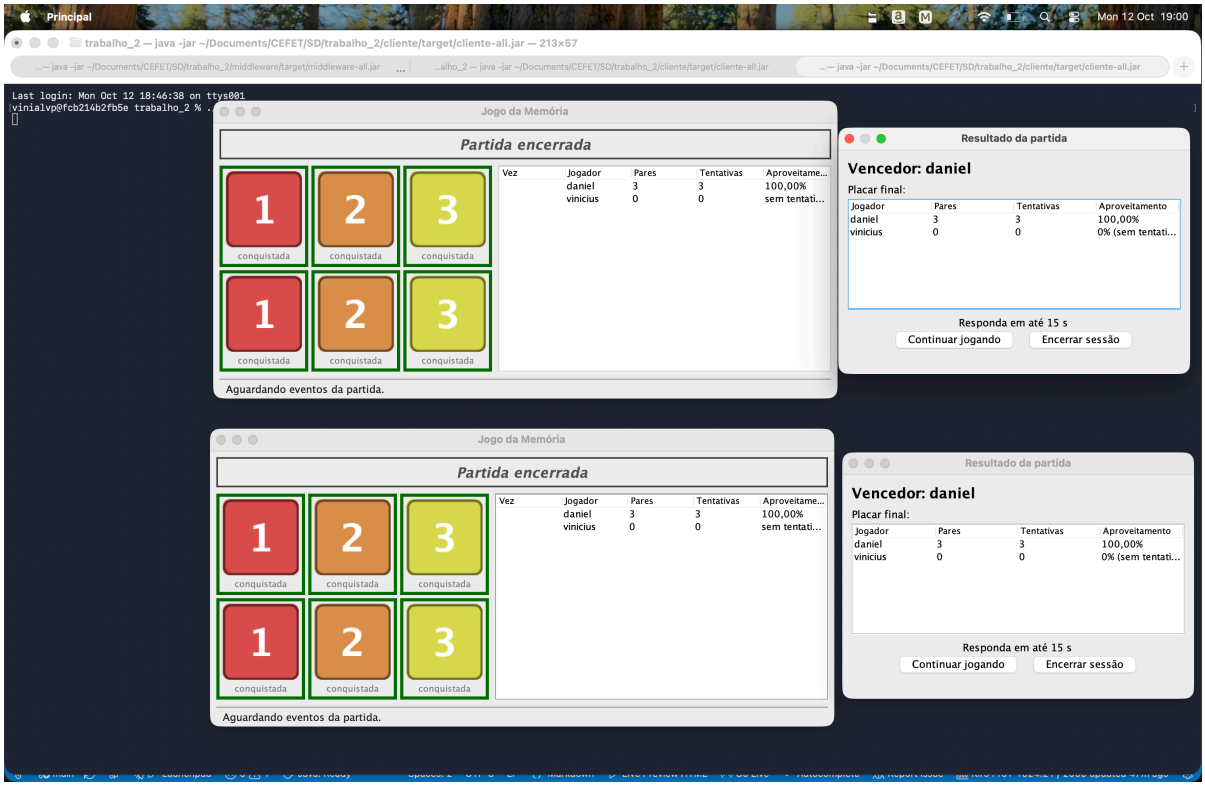


Figura 6: diálogo “Resultado da partida” nos dois clientes, com o vencedor, o placar final e os botões “Continuar jogando” e “Encerrar sessão”.

`abrirColetaContinuar` muda para `COLETA_CONTINUAR`, guarda os jogadores presentes como “esperados”, invalida temporizadores e arma o prazo de 30 s (fixo na execução normal). As regras são:

- Nova partida só se todos os esperados responderem “sim”.
- Qualquer “não” encerra a sessão na hora, sem esperar os pendentes.
- Ausência de resposta no prazo vale “não” (`PRAZO_ESGOTADO`).
- Desconexão na coleta vale “não” (`NEGATIVA_POR_DESCONEXAO`).
- A primeira resposta de cada jogador prevalece, e as demais recebem `JA_RESPONDEU`.

```
if (!continuar) { encerrarSessao(); return; }
boolean todosAceitaram = true;
for (String esperado : coleta.esperados)
    if (!Boolean.TRUE.equals(coleta.respostas.get(esperado))) { todosAceitaram = false; break; }
if (todosAceitaram && jogadores.size() >= 2) {
    temporizadores.cancelarPrazoContinuar();
    iniciarNovaPartida();
}
```

7.3 Reinício

`Sala.iniciarNovaPartida` sorteia matriz nova, fecha as cartas, zera o placar da partida (`ContadorPartida.ZERO`), incrementa o `idPartida`, volta a sequência a 1, zera a prontidão, cancela os temporizadores e escolhe o primeiro jogador pelo rodízio. Ficam preservados as partidas concluídas e canceladas, as vitórias, os pares, as tentativas e os empates da sessão. No cliente aparece "A partida foi reiniciada." e o diálogo de resultado fecha.

7.4 Tentativas e aproveitamento

Uma tentativa é uma seleção válida e completa de duas cartas, contada quando a segunda carta é aceita. Não contam o clique isolado, a seleção rejeitada e a tentativa incompleta que expira por inatividade. Um par soma par e tentativa (`comParEncontrado`), e um erro soma só a tentativa (`comErro`).

`Aproveitamento.percentual` calcula `100 × pares ÷ tentativas` em `BigDecimal`, com 2 casas e `HALF_UP`. Com 0 tentativa o valor é `0,00` com `semTentativas = true`, e o cliente mostra "0% (sem tentativas)". O acumulado da sessão soma pares e tentativas das partidas concluídas e calcula sobre as somas, não sobre a média dos percentuais:

```
jogadores.add(new SumarioJogadorDTO(e.nome(), e.vitorias(), e.pares(), e.tentativas(),
        Aproveitamento.percentual(e.pares(), e.tentativas()),
        e.tentativas() == 0, e.desconectado()));
```

Para desempate, `Aproveitamento.compararExato` compara as razões por produto cruzado (`Math.multiplyExact`), sem arredondamento.

Exemplo do enunciado:

Jogador	Vitórias	Pares	Tentativas	Conta	Aproveitamento
Ana	2	14	20	100 × 14 ÷ 20	70,00%
Bruno	1	10	25	100 × 10 ÷ 25	40,00%

Ana é campeã por vitórias (2 contra 1). Por que não a média (exemplo ilustrativo, não é execução): a partida 1 tem 5 pares em 10 tentativas (50%) e a partida 2 tem 3 em 4 (75%). A média dos percentuais dá 62,50% (incorreto), e o acumulado dá 100 × (5+3) ÷ (10+4) = 57,14%, que é o que o sistema calcula.

7.5 Sumário e campeão

`Sala.encerrarSessao` gera `SumarioSessaoDTO` (concluídas, canceladas, empates, jogadores e campeões), publica `aoEncerrarSessao` aos que restam, guarda os ids em `idsEnterrados` (que depois recebem `SESSAO_ENCERRADA`) e volta ao `LOBBY`. O sumário lista quem concluiu ao menos uma partida, e quem saiu leva `desconectado`. O campeão é o máximo lexicográfico de (vitórias, pares, aproveitamento exato). Se os três critérios empatam, há campeões empatados:

```
static int comparar(EstatisticaJogadorSessao a, EstatisticaJogadorSessao b) {
    int cmp = Integer.compare(a.vitorias(), b.vitorias());
    if (cmp != 0) return cmp;
    cmp = Integer.compare(a.pares(), b.pares());
    if (cmp != 0) return cmp;
    return Aproveitamento.compararExato(a.pares(), a.tentativas(), b.pares(), b.tentativas());
}
```

Cenário (ilustrativo)	Ana (V / P / T)	Bruno (V / P / T)	Decide	Campeão
Vitórias	2 / 10 / 20	1 / 14 / 18	Vitórias	Ana
Pares	1 / 12 / 24	1 / 10 / 12	Pares	Ana
Aproveitamento	1 / 10 / 16	1 / 10 / 20	10/16 > 10/20	Ana
Empate total	1 / 8 / 16	1 / 8 / 16	Nenhum	Ana e Bruno

Numa partida empatada, `registrarConcluida` não dá vitória, soma pares e tentativas de todos, incrementa as concluídas e guarda o grupo em `empates`. Numa partida cancelada, `registrarCancelada` só incrementa as canceladas, sem tocar estatísticas. A `TelaSumario` mostra concluídas e canceladas, o campeão ("Campeões (empate)" ou "Campeão: nenhum"), a tabela por jogador (Jogador, Vitórias, Pares, Tentativas, Aproveitamento), as partidas empatadas e o botão "Voltar à conexão", pois a volta à tela de conexão não é automática.

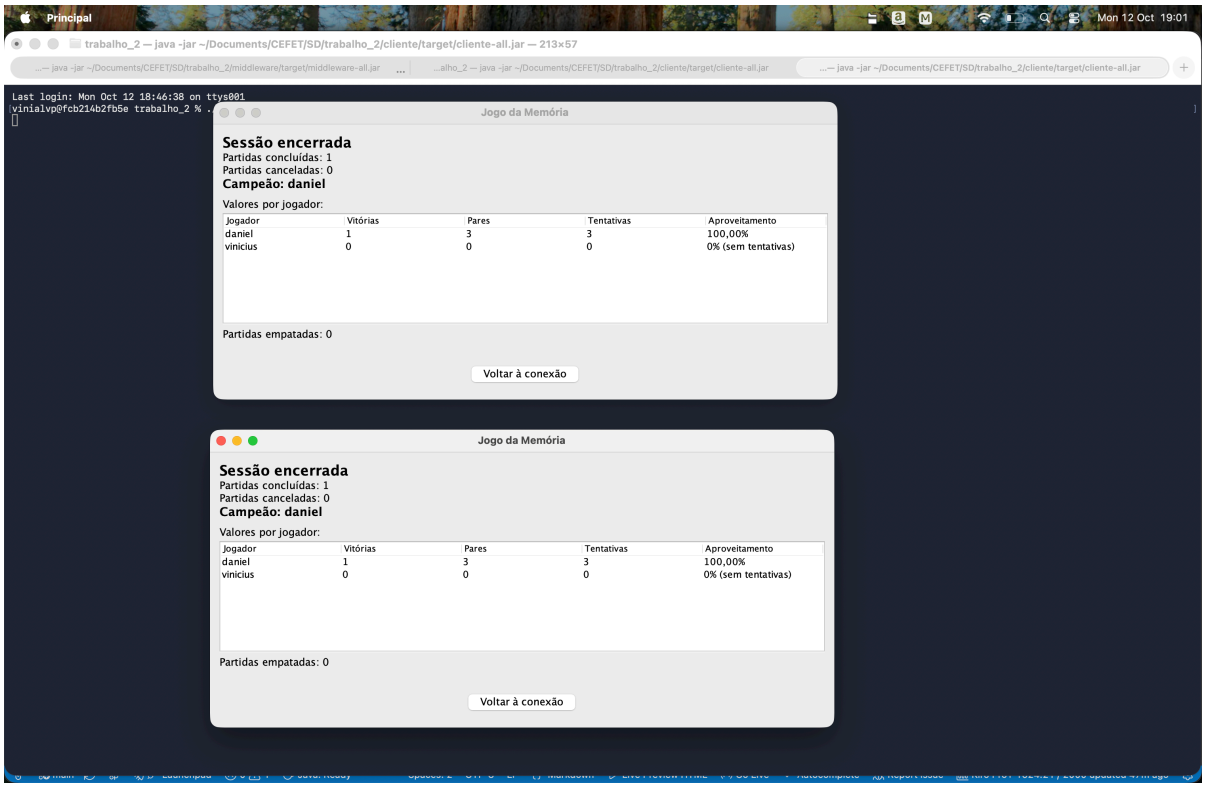


Figura 7: sumário da sessão nos dois clientes, com partidas concluídas e canceladas, o campeão e a tabela por jogador.

Na sessão das capturas (Figuras 6 e 7) houve uma partida concluída: daniel fez 3 pares em 3 tentativas ($100 \times 3 \div 3 = 100{,}00\%$) e venceu, e vinicius ficou com 0 pares e 0 tentativas (exibido como "0% (sem tentativas)"). O campeão foi daniel, com 1 vitória.

---

8. Falhas e desconexões

A política adotada é cancelar a partida afetada. A decisão ocorre sob o lock, sem I/O de rede, então um cliente indisponível não segura os outros.

8.1 Detecção

A detecção é reativa, sem heartbeat:

1. Falha ou timeout de callback no `EntregadorJogador`, com causa `REMOTE_EXCEPTION` ou `TIMEOUT` (5 s).
2. `ping()` falho na entrada, que gera a rejeição `CALLBACK_INVALIDO`, e o jogador nunca chega a existir.

3. Saída voluntária por `notificarEncerramentoCliente`, com causa `SAIDA_VOLUNTARIA` fora do lobby.

Inatividade e prazo de continuar não são desconexões. No lobby, um cliente morto pode continuar listado até a próxima entrega que falhe.

8.2 Política por fase

Em todas as fases, o jogador sai da sala e é marcado como desconectado, e os demais recebem `aoDesconectarJogador` e `aoAtualizarSala`.

Fase	Partida	Sessão e próximo passo	Restantes recebem
`LOBBY`	Nada a cancelar	Reavalia o início automático	Nada além do comum
`PARTIDA`	Cancelada uma vez, registrada à parte	Com 2 ou mais restantes, abre a coleta de "continuar" (30 s). Com 1, a sessão encerra e vai o sumário. Com 0, encerra sem notificar	`aoCancelarPartida` e `aoPerguntarContinuar` (2 ou mais) ou `aoEncerrarSessao` (1)
`COLETA_CONTINUAR`	Nada a cancelar	Vale "não" e a sessão encerra	`aoEncerrarSessao`

Partida cancelada não conta como concluída e não entra nas estatísticas, e o sumário a mostra à parte. A decisão está em `Sala.tratarDesconexao`, que copia a fase antes de remover o jogador:

```
FaseSala faseNaDesconexao = fase;
removerDesconectado(j, causa);
switch (faseNaDesconexao) {
    case LOBBY -> avaliarInicioAutomatico();
    case PARTIDA -> tratarDesconexaoEmPartida(j, causa);   // cancela; >= 2: coleta; senao encerra
    case COLETA_CONTINUAR -> tratarDesconexaoNaColeta(j);  // vale "nao"; encerra
    default -> { }
}
```

8.3 Idempotência e isolamento

O entregador notifica uma vez. O `tratarDesconexao` ignora id desconhecido ou já desconectado, o `tratarDesconexaoEmPartida` só age com a partida `EM_ANDAMENTO`, e o lock serializa falhas de workers diferentes. Com 3 ou mais jogadores, duas falhas seguidas podem levar a pergunta de continuar seguida do sumário, o que é coerente com a regra.

Optou-se por cancelar em vez de pausar porque não há reconexão (a entrada só ocorre no lobby, com novo id), e a pausa precisaria de prazo e acabaria em cancelamento. O grupo decide então se continua.

O isolamento vem de um worker e uma fila por jogador, do timeout de 5 s por chamada, da fila descartada na falha e do lock, que nunca é segurado durante I/O (o `ping` é o único I/O que o chamador espera, fora do lock). Uma ressalva é que `cancel(true)` interrompe quem espera, mas uma chamada presa em socket pode ocupar uma thread do pool, que cresce sob demanda. Isso não foi medido.

8.4 Cliente diante de falhas

Situação	Comportamento
Falha antes de entrar	Volta à tela de conexão, com host e porta no erro
Rejeição de domínio	Mostra o motivo, sem mudar o tabuleiro
Timeout de seleção	Mantém o tabuleiro
`RemoteException`	Marca `conexaoPerdida`, desabilita o jogo (`VisaoTabuleiro`) e abre o diálogo "Conexão perdida" (Voltar à conexão ou Permanecer aqui). A perda só é percebida ao fazer uma chamada
Falha de consulta	`falhaSincronizacao` desabilita seleções
Fechar a janela	Chama `notificarEncerramentoCliente` (espera até 2 s) e encerra a JVM

8.5 Log

O log tem uma linha por evento, no console e, com `-Dmemoria.log=caminho`, em arquivo (append, UTF-8), no formato `yyyy-MM-dd HH:mm:ss.SSS | TIPO | jogador | resultado | detalhes`. Os tipos são `CONFIG`, `ENTRADA`, `SAIDA`, `PRONTO`, `INICIO_PARTIDA`, `JOGADA` (`ACEITA` ou `REJEITADA`), `TROCA_DE_TURNO`, `RESULTADO`, `CONTINUAR`, `SUMARIO`, `DESCONEXAO`, `CANCELAMENTO` e `ERRO`. O exemplo abaixo mostra o formato e não é uma execução real:

```
... | DESCONEXAO | Bruno | TIMEOUT | fase=PARTIDA restantes=2
... | CANCELAMENTO | Bruno | TIMEOUT | partida=1 sequencia=14 restantes=2 canceladas=1
... | CONTINUAR | - | ABERTA | esperados=2 prazoSeg=30
```

---

9. Conclusão, limitações e referências

9.1 Conclusão

O sistema entregue implementa um jogo da memória multijogador com um middleware central em Java e clientes Swing independentes, que se comunicam por Java RMI. O middleware é a única autoridade: guarda a matriz secreta, valida cada jogada sob um lock único, controla turnos, placar e sessão, e publica o estado por callbacks, de modo que o cliente só recebe as cartas reveladas. Reinício de partida, estatísticas de sessão, aproveitamento e escolha do campeão são calculados no servidor, e o cliente apenas exibe o resultado.

As principais decisões foram serializar todo o estado oficial em um lock justo por sala, entregar os callbacks fora do lock por uma fila e um worker por jogador, numerar os eventos com a sequência de estado para que o cliente descarte notificações antigas, usar um token de época para ignorar temporizadores obsoletos, e cancelar a partida afetada em vez de pausá-la quando um jogador cai. As limitações estão na seção 9.2.

9.2 Limitações

O sistema não tem autenticação nem criptografia. O `idParticipante` é a única credencial e é conhecido por todos os clientes, então um cliente modificado poderia agir por outro jogador, e o uso deve ficar restrito a uma rede confiável. O estado fica só em memória e há uma sala por middleware, de modo que reiniciar o servidor descarta tudo. A entrada só é possível no lobby, sem reconexão nem pausa, e uma partida cancelada não é retomada.

Há também limites de infraestrutura. O callback e os objetos remotos usam porta anônima, o que exige liberar o firewall nas duas pontas, e `rmi.portaObjetos` e os prazos não são configuráveis pelo `main`. A fila de eventos por jogador não tem limite e a detecção de falhas é reativa, sem heartbeat, então um cliente morto só é percebido na próxima entrega que falhar. A interface exige ambiente gráfico, os scripts exigem Bash (os jars rodam com `java -jar`) e não há testes automatizados.

Na interface, o cliente não conhece o sub-estado da partida: durante o intervalo de erro as cartas fechadas continuam habilitadas e o servidor rejeita o clique. Além disso, `falhaSincronizacao` não tem caminho de recuperação na tela, e, se a sincronização inicial estourar sem perda de conexão, as seleções ficam desabilitadas até reconectar. O sumário só lista quem concluiu uma partida, e, se todas foram canceladas, mostra “Campeão: nenhum”.

9.3 Código-fonte

O código-fonte e os scripts estão em https://github.com/tinicius/trabalho-2-sd.

O projeto tem os módulos `comum/` (contrato e catálogo), `middleware/` e `cliente/`, além de `scripts/` e `executaveis/`. As imagens ficam em `comum/src/main/resources/images/` (`par-01.png` a `par-12.png`, `verso.png` e `catalogo.txt`). Os pontos de entrada são `middleware.Principal`, `middleware.servico.PublicadorRmi`, `middleware.sala.Sala`, `middleware.nucleo.Partida`, `ValidadorJogada`, `GeradorMatriz`, `CalculadoraCampeao`, `cliente.AplicacaoCliente`, `cliente.rmi.CallbackJogadorImpl` e `cliente.modelo.ModeloCliente`. Os passos de instalação e execução estão no `README.md` do repositório.

9.4 Referências

- Enunciado do Trabalho 2 de Sistemas Distribuídos (CEFET).

- Documentação do Java SE 21: https://docs.oracle.com/en/java/javase/21/
- Java RMI, especificação (Java SE 21): https://docs.oracle.com/en/java/javase/21/docs/specs/rmi/index.html
- Filtros de serialização (`ObjectInputFilter`), Java SE 21: https://docs.oracle.com/en/java/javase/21/core/serialization-filtering1.html
- Tutorial de Swing (Java Tutorials): https://docs.oracle.com/javase/tutorial/uiswing/
- Apache Maven, guias: https://maven.apache.org/guides/
- Documentação do Mermaid, usado nos diagramas: https://mermaid.js.org/